Exploring Large Language Models

02

CSG202 · Module 2







Learning objectives

Define an LLM

Explain how large language models learn and generate text.

Trace the data flow

Connect tokens, embeddings, Transformers, and inference.

Match use cases

Recognize suitable tasks, limitations, and validation needs.

Use prompting concepts

Distinguish prompt design, prompt tuning, and fine-tuning.

Module topics



01



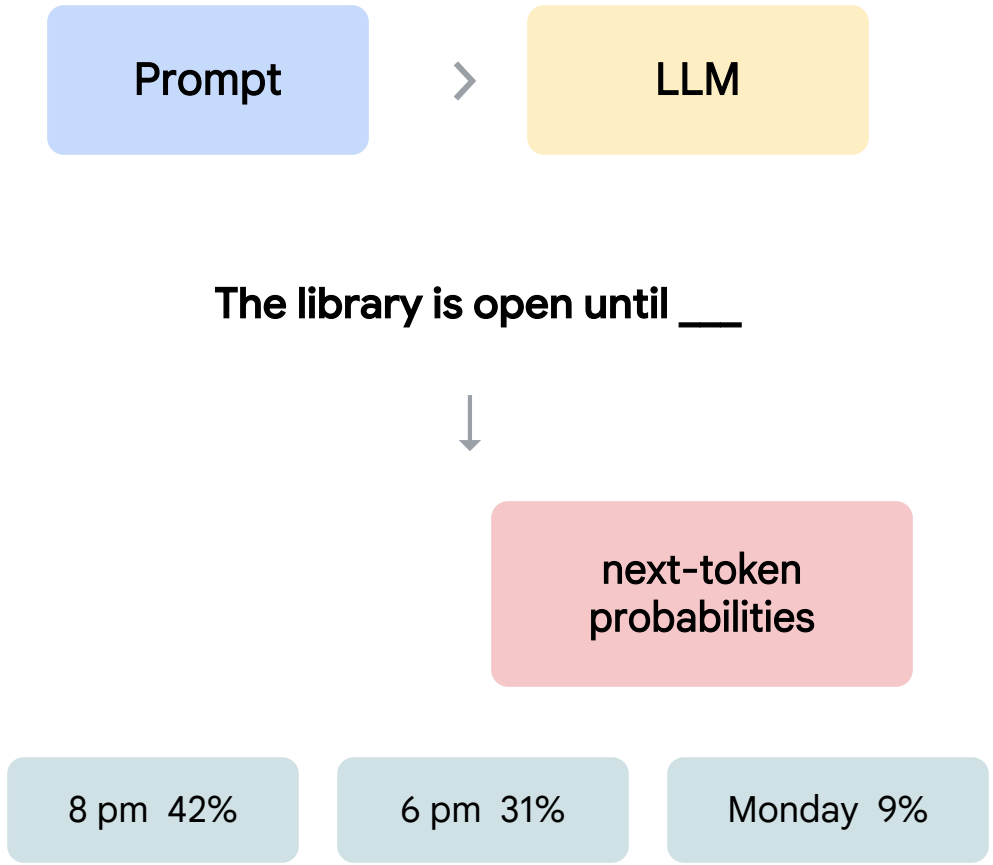
Large language models

Definition, scale, evolution, and model families

An LLM predicts language from learned patterns

A language model estimates which token is likely to come next in a sequence.

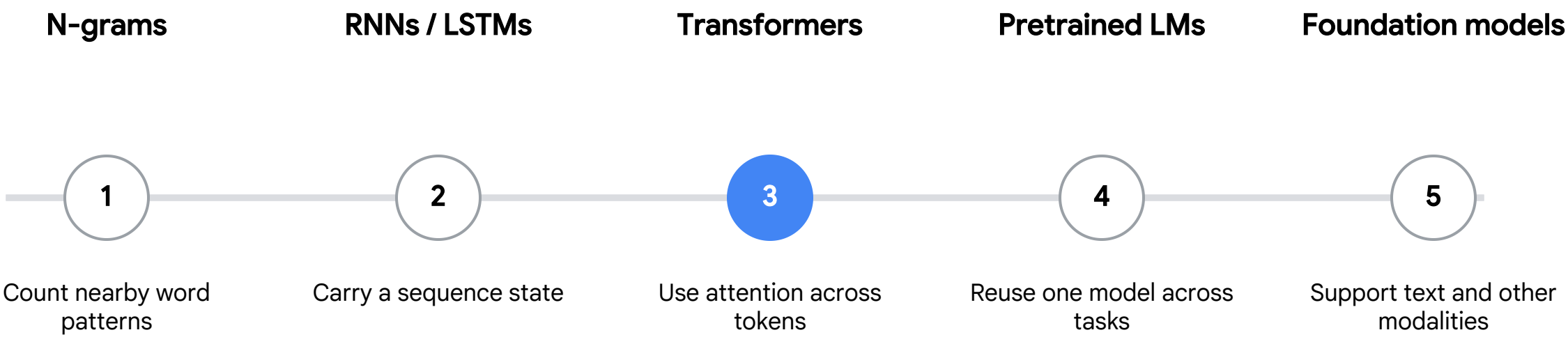
- "Large" usually refers to model capacity, training data, and compute—not only parameter count.
- The model stores statistical patterns in learned numerical weights, not a searchable copy of every source.
- Generating one token repeatedly can produce paragraphs, code, summaries, or dialogue.



Traditional language systems and LLMs solve different shapes of problems

Compare	Traditional NLP system	Large language model
Typical scope	One narrow task	Many language tasks
Data and features	Often task-specific	Broad pretraining, then prompting or adaptation
Changing behavior	Rewrite rules or retrain	Change prompt; ground, tune, or retrain if needed
Example	Spam classifier	Assistant that summarizes, translates, and answers questions

LLMs evolved by learning longer and richer relationships



The Transformer architecture made it practical to model relationships between many tokens in parallel.

02



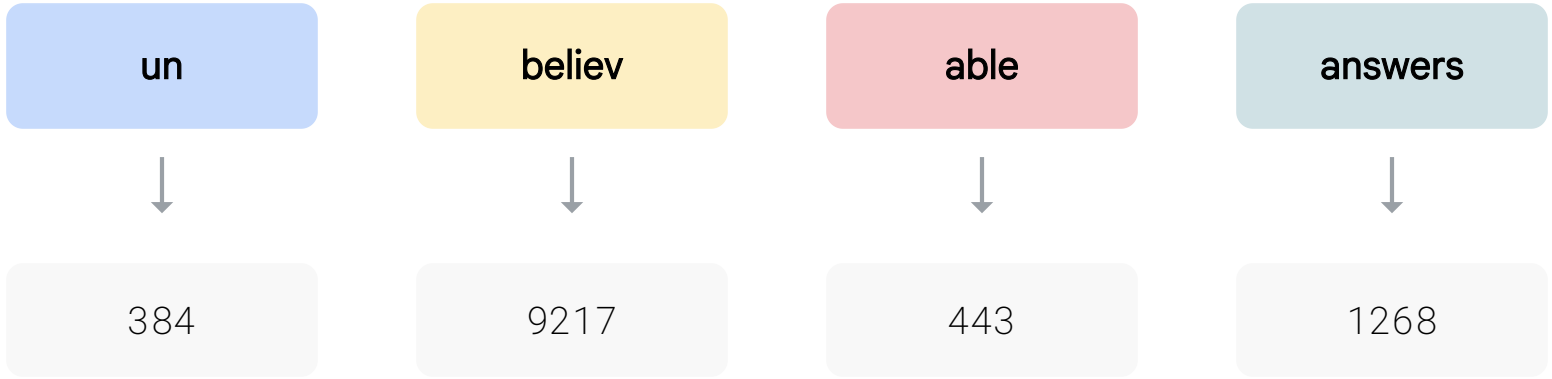
Inside an LLM

Tokens, embeddings, Transformers, training, and inference

Tokenization converts text into model-readable pieces

“unbelievable answers” › un | believ | able | answers

Tokens may be words, word pieces, punctuation, or spaces. The exact split depends on the tokenizer.

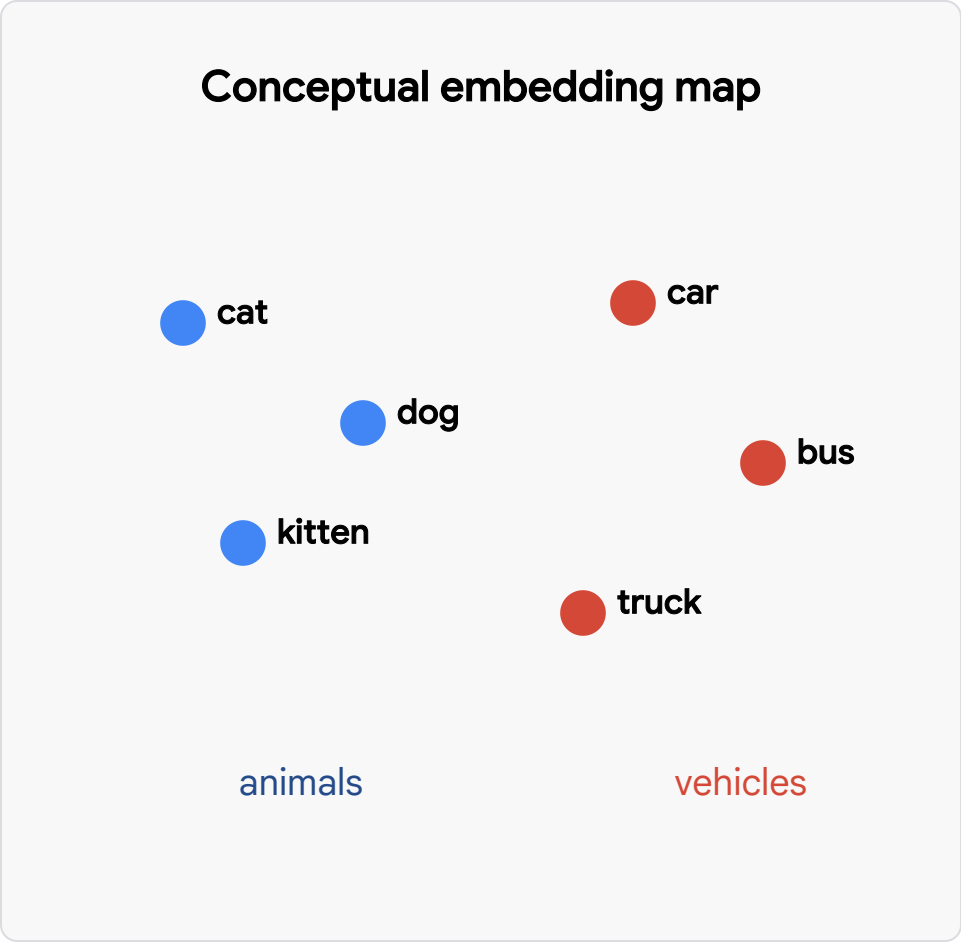


Token IDs are tokenizer-specific catalog numbers—not meanings.

Embeddings place tokens in a learned numerical space

An embedding is a vector: a list of numbers that represents a token or input for the model.

- Tokens used in similar contexts often receive related representations.
- Modern models update representations as context moves through the layers.
- The two-dimensional picture is only an analogy; real embeddings use many dimensions.

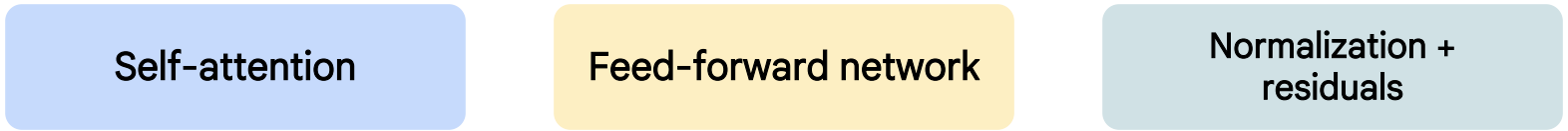


Transformers combine token meaning with position and context

“dog bites person” and “person bites dog” contain similar tokens—but position changes the meaning.

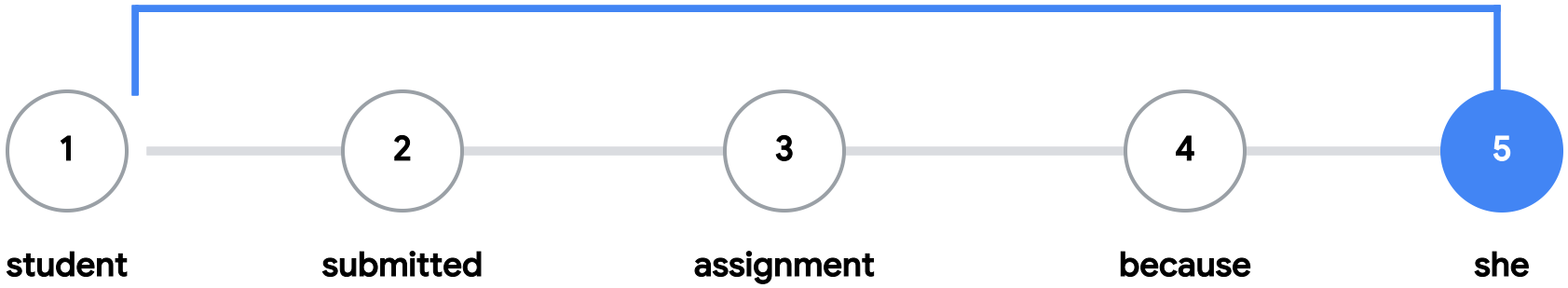


Inside each block



Self-attention asks which earlier tokens matter for this token

The student submitted the assignment because she finished early.



The representation for “she” can give more weight to “student” than to unrelated tokens.

Attention weights are learned and context-dependent; they are not a human explanation of every decision.

Transformer families arrange encoders and decoders differently

Encoder-only

Reads the full input and builds representations

Typical tasks

Classification, search, extraction

BERT-style

Encoder–decoder

Encoder reads; decoder generates an output

Typical tasks

Translation, summarization

T5-style

Decoder-only

Predicts the next token autoregressively

Typical tasks

Chat, writing, code generation

GPT-style LLMs

Commercial model details may be partly unpublished; classify by observed behavior and official documentation.

A decoder-only Transformer repeats the same block across many layers

Input preparation

Prompt tokens

↓

Token embeddings

+

Position information

The model now has token meaning and order.

Decoder block × many layers

Masked multi-head self-attention

Each position can use earlier context—not future generated tokens.

↓

Add residual + normalize

↓

Feed-forward network

↓

Add residual + normalize

Output preparation

Contextual token representations

↓

Vocabulary scores

↓

Softmax probabilities

Select one token, append it, and run the block again.

A user prompt becomes one generated token at a time

User prompt: “Summarize: The campus library will close at 8 PM during exams.”

1

Tokenize

Split the prompt into token IDs

2

Embed + position

Represent meaning and order

3

Run decoder layers

Attention mixes relevant earlier context

4

Rank next tokens

Projection + softmax create probabilities

5

Select + append

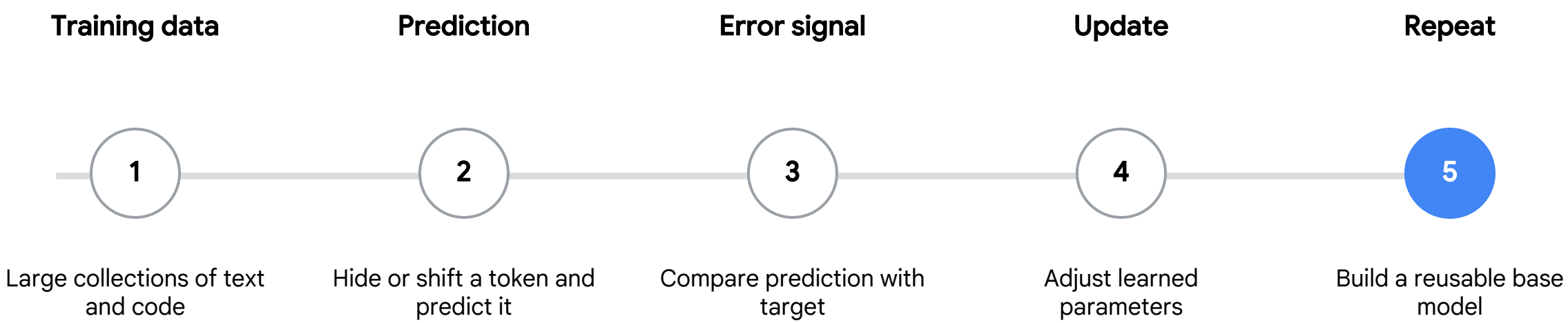
Add one token and repeat

While producing “8”, masked self-attention can emphasize earlier tokens such as “library”, “close”, and “8 PM”.

Illustrative next-token ranking: 8 (highest) · eight (possible) · 9 (unlikely) → append “8” and repeat

The model recomputes context for every new token; it does not retrieve facts unless the application supplies a search or retrieval tool.

Pretraining teaches a base model by repeated prediction



The result is a general model—not yet a safe, reliable application.

Training learns parameters; inference uses them

Training

Many examples

Predict + measure error

Update parameters

High compute and time

Inference

One request or batch

Use stored parameters

Generate or score output

Milliseconds to minutes

Prompting happens during inference; tuning is a training activity.

Text generation repeats one next-token decision

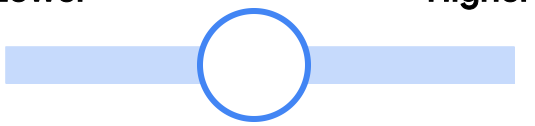
1. Read prompt
2. Compute scores
3. Convert to probabilities
4. Select a token
5. Append and repeat

Prompt: “A safe password should…”
Generated sequence: use → multiple → unique → characters → …

The loop stops at a stop sequence, end token, tool call, or output limit.

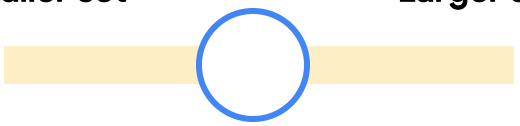
Generation controls shape variation, length, and usable context

Temperature



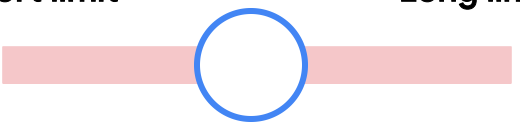
More predictable — More varied

Top-p



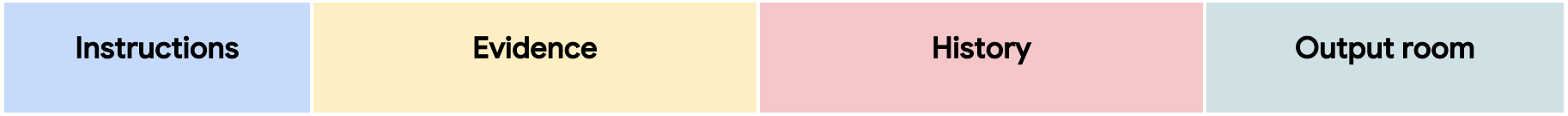
Conservative choices — More alternatives

Max output tokens



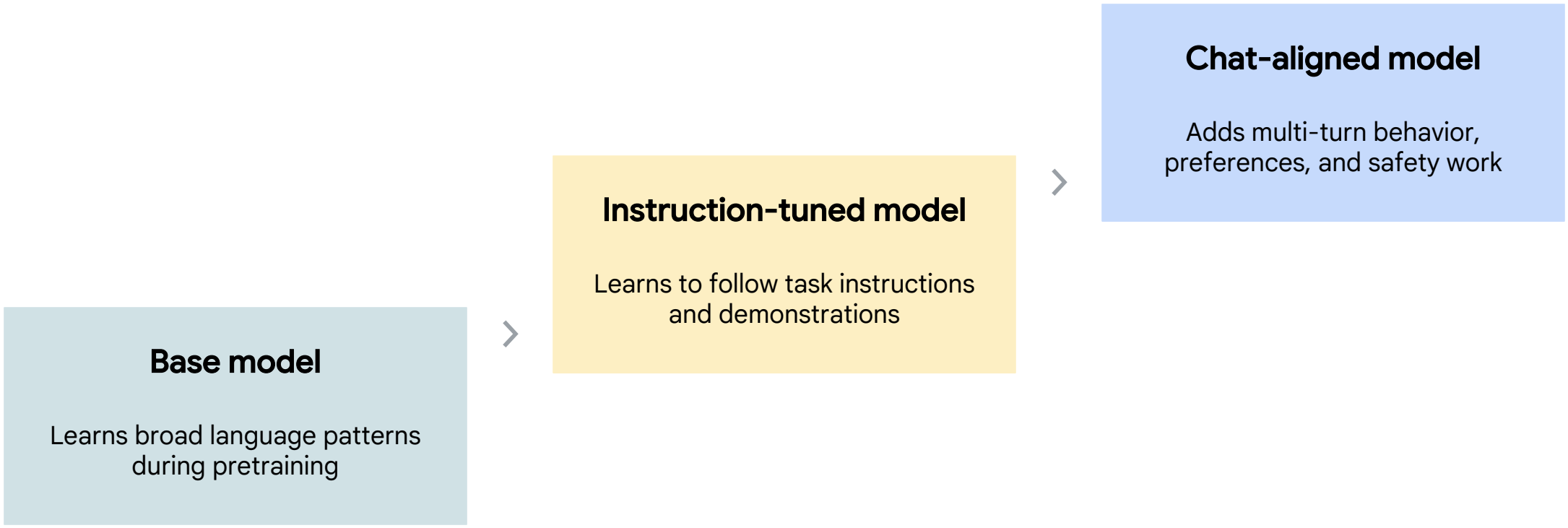
Brief response — More room—not more truth

Context window = instructions + examples + retrieved evidence + conversation + requested output



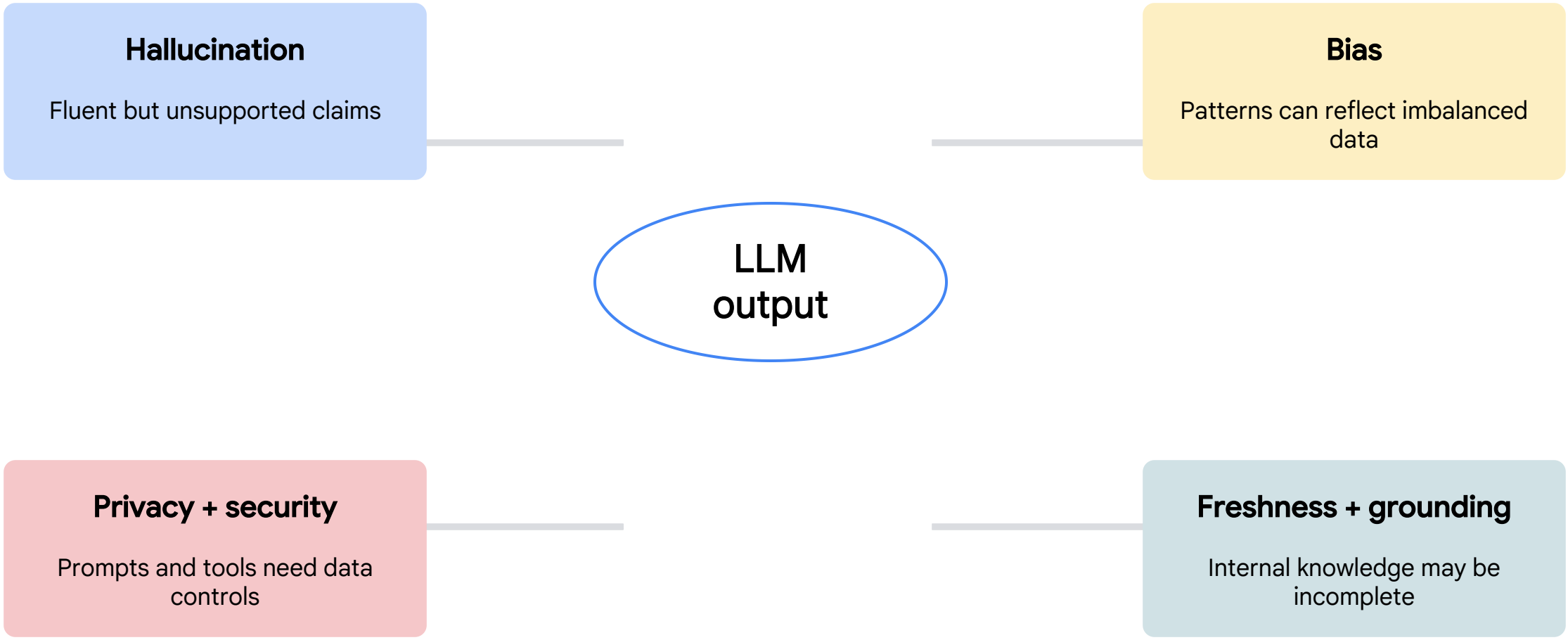
A larger context window does not guarantee perfect recall or equal attention to every detail.

Base, instruction, and chat models represent stages of adaptation



Alignment improves useful behavior—but it does not remove hallucination, bias, or security risks.

LLM output is useful only when its limitations are managed



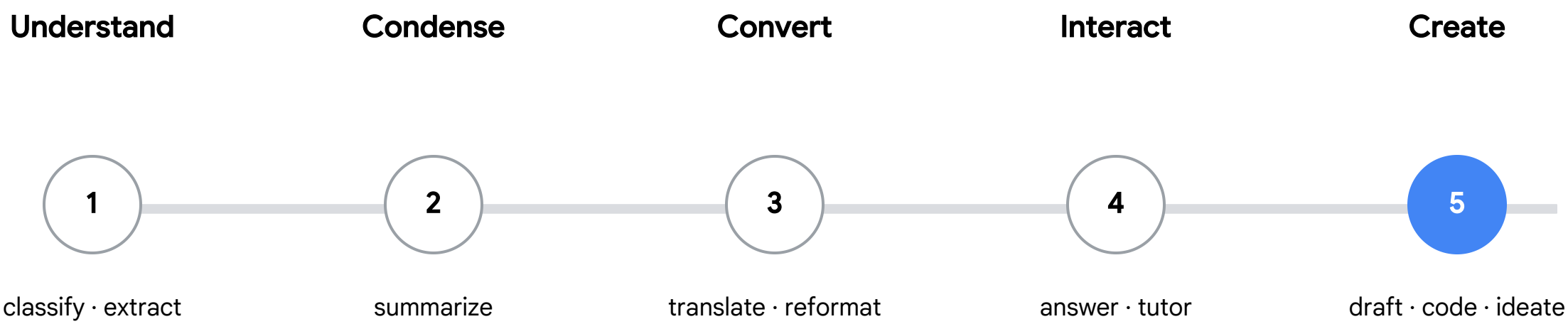
Verify important claims against trusted evidence and define when a human must review the output.

03

LLM use cases

Match the model to the task, data, risk, and evaluation method

LLM applications range from transforming content to creating it



The same LLM can support several tasks; the prompt, evidence, controls, and evaluation define the application.

Four core use cases connect an input to a useful output

1 Summarization

Input

10-page paper

↓

Output

3-sentence abstract

2 Translation

Input

English product page

↓

Output

Hindi version with terms preserved

3 Coding

Input

Requirement or error

↓

Output

Draft code + explanation

4 Conversational AI

Input

Student question + policy

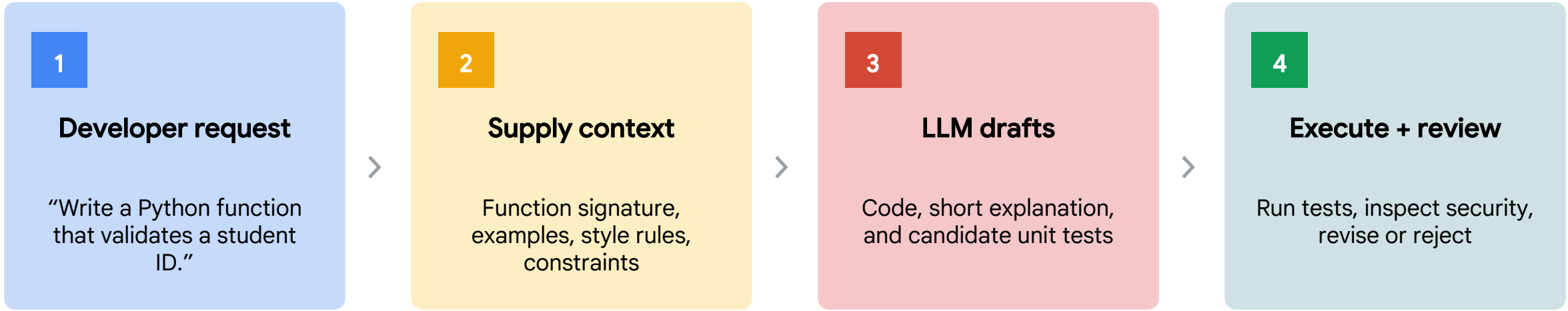
↓

Output

Context-aware answer

For every use case, define success before selecting the model.

Example: a coding assistant turns requirements into a reviewable draft



The model accelerates drafting; executable tests and human review determine whether the code is usable.

Evaluate an LLM application with task-specific evidence

1

Task success

Is the answer complete, relevant, and in the required format?

2

Grounded accuracy

Are important claims supported by allowed evidence?

3

Consistency

Does quality remain acceptable across representative prompts?

4

Operational fit

Are latency, cost, safety, and escalation acceptable?

Define these checks before comparing prompts, models, or tuning methods.

04

Prompting and tuning

Guide a model with instructions, examples, evaluation, and adaptation

A strong prompt gives the model task, context, examples, and constraints

1	**Task**	Summarize the policy for a first-year student.
2	**Context**	Use only the policy text enclosed below.
3	**Examples**	Show one acceptable summary format.
4	**Constraints**	Five bullets; plain language; cite section numbers.

System instructions can set persistent role, style, and rules across several prompts.

Examples change how clearly the model can infer the expected pattern

Zero-shot	One-shot	Few-shot
Instruction only	One example	Several examples
`Classify: "Delivery was late."`	`"Loved it" → Positive` `Now classify: "Delivery was late."`	`Positive, neutral, and negative examples` `Then classify the new review`
No demonstration	**Shows one pattern**	**Clarifies boundaries**

More examples are useful only when they are representative, consistent, and within the context budget.

Prompt design, engineering, tuning, and fine-tuning change different things

Compare	Prompt design	Prompt engineering	Prompt tuning	Fine-tuning
Main action	Write one clear prompt	Test and improve prompts systematically	Learn soft prompt vectors	Learn model parameters or adapters
What changes?	Prompt text	Prompt text + workflow	Small learned vectors; base model frozen	Selected or all model parameters
When useful	Single task	Repeated prompt workflow	Stable task with training examples and support	Specialized behavior with strong data
First step?	Yes	Yes	After prompting and evaluation	After prompting and evaluation

Start with prompt design and evaluation; adapt the model only when a measured problem remains.

Prompt tuning learns a small task-specific input for a frozen model



What prompt tuning is

- A training method
- Compact learned vectors
- Reusable for a stable task

What prompt tuning is not

- Rewriting natural-language prompts
- Adding one example manually
- Guaranteed support for every model

Check current model support before planning a tuning activity.

Improve the application in the least expensive order

1

Prompt

Clarify task, context, examples, and output format

2

Ground

Add trusted current evidence or tools

3

Evaluate

Measure recurring errors on a test set

4

Tune if needed

Use supported tuning with high-quality examples

Every stage needs the same evaluation set so improvements can be compared fairly.

A better prompt makes the expected output observable

Weak prompt

"Explain cloud security."

›

Improved prompt

```
Task: Explain shared responsibility.
Audience: First-year cloud students.
Context: Use the supplied course
notes only.
Output: 4 bullets + one campus
example.
Constraint: Flag anything not
supported by the notes.
```

Quality becomes testable: correct concept, beginner language, four bullets, relevant example, and evidence discipline.

05

Google Gen AI development tools

Discover, prototype, integrate, evaluate, and practise in Google Skills

Product names change; the development jobs remain familiar

Labels students may see	Durable role in the workflow
Vertex AI / Generative AI on Vertex AI	Managed platform for building and operating AI applications
Vertex AI Studio / Studio tools	Interactive prompt and model prototyping
Model Garden	Model discovery, comparison, deployment options, and model cards
Gemini API + Google Gen AI SDK	Programmatic integration into applications

Before a live class, verify current names, regions, model versions, preview status, and retirement dates.

Choose a model family by workload—not by the newest name

Workload need	Typical choice	Why	Example
Complex reasoning	Pro / reasoning tier	Highest capability; usually slower and more expensive	Multi-step analysis or difficult coding
General application	Flash / balanced tier	Strong quality with lower latency and cost	Assistant, summarization, extraction
High-volume simple task	Flash-Lite / efficient tier	Optimized for throughput and cost	Classification or short structured output
Open customization	Gemma open models	Downloadable weights under the relevant license	Research or controlled deployment

Version numbers and availability change rapidly—confirm the current official model catalog before delivery.

Studio tools and Model Garden answer different questions

Studio tools

“How does this model respond?”

- Write and test prompts
- Set system instructions and generation controls
- Compare responses
- Generate starter code

Best for: rapid prototyping

Model Garden

“Which model should we consider?”

- Browse Google, partner, and open models
- Read model cards and capabilities
- Compare modality, license, and deployment
- Start supported customization or deployment

Best for: model discovery

Open the Google tool that matches the immediate task

1

Find a model

Model Garden
model cards + options

2

Test a prompt

Studio tools
prompts + controls

3

Call from code

Gemini API
Google Gen AI SDK

4

Practise safely

Google Skills
temporary lab credentials

This module identifies each tool's purpose. Deployment, monitoring, governance, and multimodal workflows are developed in later modules.

Current product names and access methods should be verified before every live class.

Google Skills lab

Design Prompts in Agent Platform

GSP1151 · about 30 minutes · introductory

Temporary lab credentials
Private/Incognito browser
Complete notebook tasks and progress checks
Lab timer cannot be paused

LAB

Prompt engineering best practices

Reduce variability

Improve responses with examples



Review: Exploring large language models

1	**LLM**	Predicts tokens using learned patterns; “large” includes model, data, and compute.
2	**Inside**	Text → tokens → embeddings → Transformer → probabilities → generated tokens.
3	**Use cases**	Summarize, translate, code, converse, extract, classify, tutor, and search—with evaluation.
4	**Prompting**	Task, context, examples, and constraints shape output; prompt tuning is a training method.
5	**Google tools**	Model Garden discovers; Studio prototypes; APIs/SDKs integrate; Google Skills enables labs.

Activity

Think · Pair · Share

Choose one campus task that could benefit from an LLM.

Define the input, expected output, success measure, and one risk.

Decide whether you would begin with prompting, grounding, or tuning—and explain why.

Use 3 minutes individually, 4 minutes in pairs, and 1 minute for two pairs to share.



Knowledge check

Three questions to confirm the key distinctions

Question #1: What happens during inference?

A. The training dataset is collected

B. Model parameters are initialized

C. A trained model generates an output for an input

D. Every parameter is updated

Answer #1

Correct answer: C. A trained model generates an output for an input

Training updates parameters. Inference uses the trained parameters to produce predictions or generated content.

Question #2: Which statement best describes prompt tuning?

A. A human rewrites the prompt until it looks clearer

B. The model searches the web before answering

C. A small set of soft prompt vectors is learned while the base model remains frozen

D. Every model parameter is retrained from scratch

Answer #2

Correct answer: C. Soft prompt vectors are learned while the base model remains frozen

Prompt design edits human-readable text. Prompt tuning learns compact task-specific vectors and requires supported training infrastructure.

Question #3: Which Google tool is primarily used to discover and compare models?

A. Model Garden

B. A spreadsheet

C. A prompt temperature control

D. A context window

Answer #3

Correct answer: A. Model Garden

Studio tools help test prompts; Model Garden helps discover and compare Google, partner, and open models.

References: LLMs, Transformers, and prompting

Google Machine Learning Crash Course — Large language models

Beginner overview of LLMs, parameters, prompting, and inference.
https://developers.google.com/machine-learning/crash-course/llm

Google Machine Learning Crash Course — Transformers

Self-attention and the Transformer architecture.
https://developers.google.com/machine-learning/crash-course/llm/transformers

Attention Is All You Need

Original Transformer research paper.
https://arxiv.org/abs/1706.03762

Introduction to prompting — Google Cloud

Prompt components, prompt design, and prompt engineering.
https://cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/introduction-prompt-design

Overview of prompting strategies — Google Cloud

Prompt structure, examples, iteration, and evaluation.
https://cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/prompt-design-strategies

References: tuning, Google tools, and lab

Introduction to tuning — Google Cloud

Prompting versus tuning and currently supported tuning approaches.
https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/tune-models

Generative AI on Vertex AI — Google Cloud

Current platform overview and featured capabilities.
https://docs.cloud.google.com/vertex-ai/generative-ai/docs

Model Garden — Google Cloud

Discover, evaluate, customize, and deploy supported models.
https://docs.cloud.google.com/vertex-ai/generative-ai/docs/model-garden/explore-models

Google Gen AI SDK — Google AI for Developers

Current developer libraries for Gemini access.
https://ai.google.dev/gemini-api/docs/libraries

Google Skills lab GSP1151

Design Prompts in Agent Platform; temporary credentials and guided tasks.
https://www.skills.google/focuses/63251